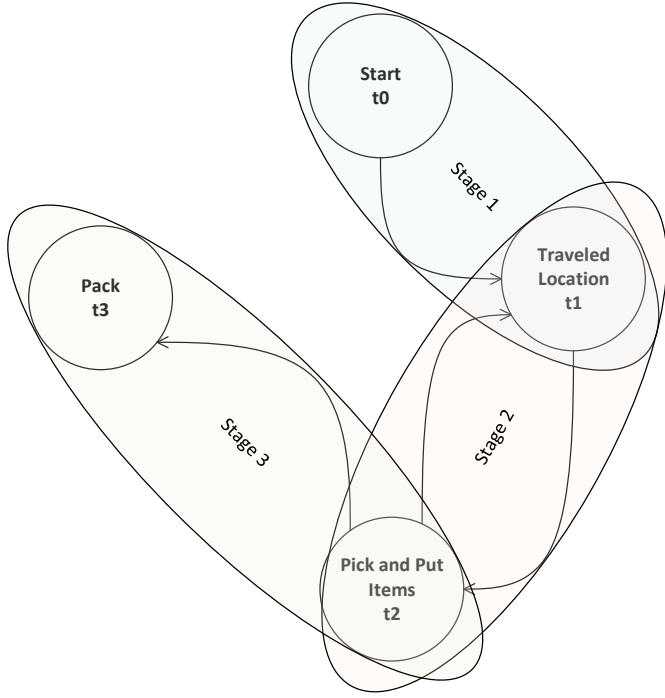


Orders Pick and Pack Performance Analysis

Traveled Time Performance Based on Pick and Pack Stages



Total time Traveled States (TS_t):

$$TS_t = t_{(0,1)} + t_{(1,2)} + t_{(2,3)}$$

$$t_1 =$$

$$\sum_{i=n}^{i=0} tLoc_i$$

$$t_2 =$$

$$\sum_{i=n}^{i=0} tI_i$$

$$t_{(1,2)} = \sum_{i=n}^{i=0} tLoc_i + tI_i$$

$tLoc_i$ = time to travel to a location, where $i = n$ is the last location.

tI_i = time to pick & put an item, where $i = n$ is the last item.

Stage → S | Real Time Data

Current Value for Stages 1, 2, and 3

S1	S2	S3
$t_{(0,1)}$	$t_{(1,2)}$	$t_{(2,3)}$

Overall Value for all Possible Stages

$$\bigcup_{X=3}^{X=0} SX = \frac{\sum_{i=n}^{i=0} t_{(x, x+1)i}}{n}$$

Stage → S | Approximated/Predicted Data

Average Value for Stages 1, 2, and 3 based on prior data

$S1_{avg}$	$S2_{avg}$	$S3_{avg}$
$\sum_{i=n}^{i=0} t_{(0,1)i}/n$	$\sum_{i=n}^{i=0} t_{(1,2)i}/n$	$\sum_{i=n}^{i=0} t_{(2,3)i}/n$

Overall Average Value for all Possible Stages

$$\bigcup_{X=3}^{X=0} SX_{avg} = \frac{\sum_{i=n}^{i=0} t_{(x, x+1)i}}{n}$$

U	S1	S2	S3
S1	1	0	0
S2	0	1	0
S3	0	0	1
TS	1	1	1

$$\therefore TS_t = S1 + S2 + S3$$

Performance Indicator:

$$TS_p = 100\% - \left(\frac{\sqrt{(TS_t + \Delta TS_t)}}{\Delta TS_t} \right)$$

TS_p = Traveled Stages Performance in %

R_{exp} = Individual Exponential Growth Rate

PIR_{avg} = Overall Exponential Growth Rate

$$\Delta TS_t = S1_{avg} + S2_{avg} + S3_{avg}$$

Performance Indicator Rate (PIR):	Average PIR
R_{exp} $= \frac{\sqrt{(TS_p^2 + \Delta TS_p^2)}}{\Delta TS_p}$	$PIR_{avg} = \frac{\sum_{i=n}^{i=0} R_{exp_i}}{n}$

For a 100% Individual Performance Rate

$$R_{exp} = \frac{\sqrt{(100^2 + 100^2)}}{100} = 1.4142 \approx 1.4$$

For a 100% Overall Performance Rate

$$PIR_{avg} = \frac{\sum_{i=n}^{i=0} 1.41}{n}$$

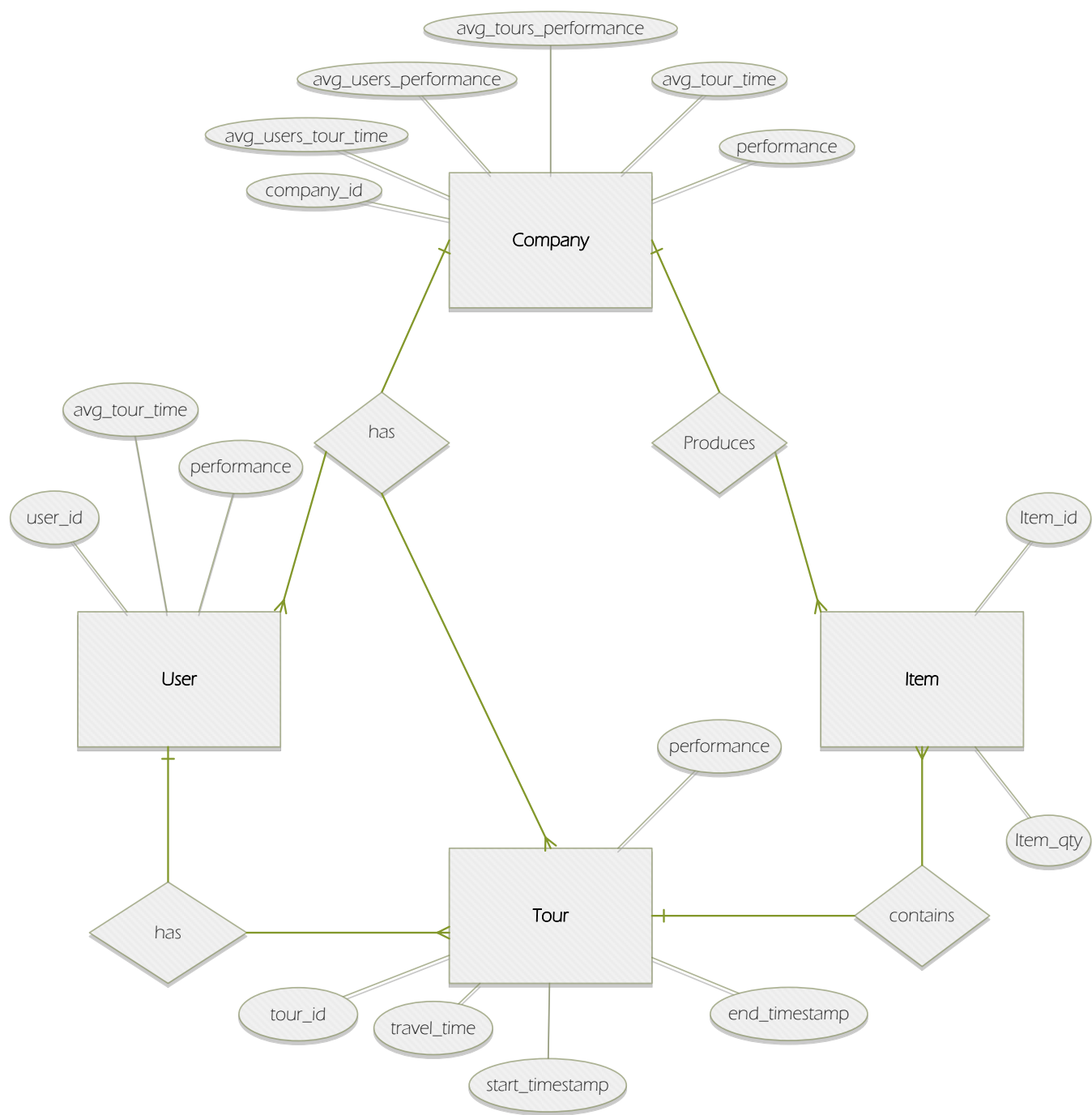
Data Field for Time Study Analysis

Main Data Fields			Data Fields Description			Data Fields Values		
Stage ₁	Stage ₂	Stage ₃	State _{0,1}	State _{(1,2)_i} → i = {0, ..., n}	State _(2,3)	t ₀ ⁺ tLoc ₀	tLoc _i ⁺ tI _i	tLoc _n ⁺ t ₃

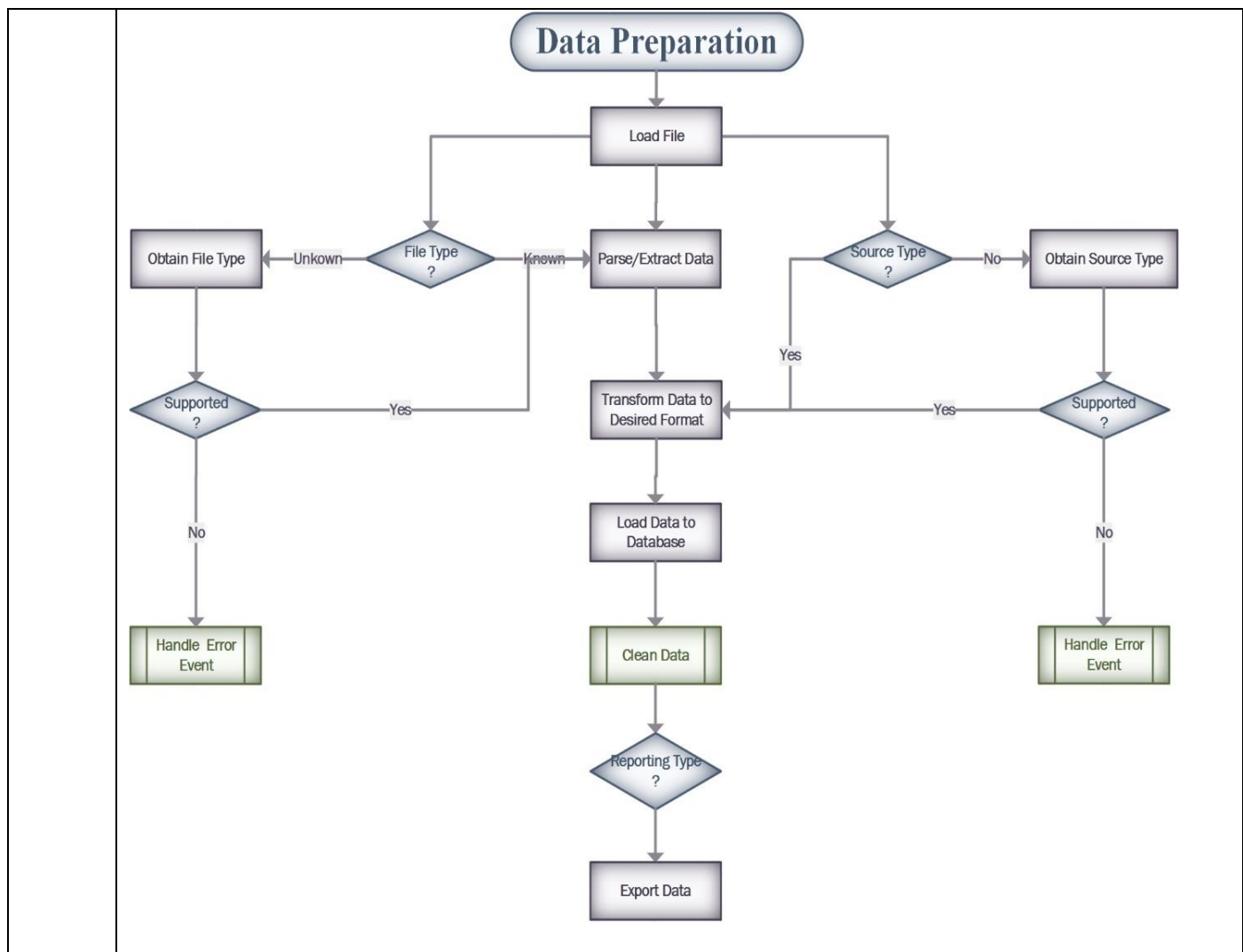
Pick & Put Algorithm	
Approximate traveled time Start-point (loc_0) - Scan location loc_0, where $i = 0$ is the Start-point - Record stamp-time - Got to next_loc = loc_1, where next location is equal to $loc_{(i+1)}$ - Loop from next_loc to last location loc_n, where n= last location - Travel to location (loc_i), where $i = \{1, \dots, n\}$ - Scan location loc_i, where $i = n$ is the last location. - Record stamp-time - Calculate traveled-time between $loc_{(i-1)}$ to loc_i (tt_d) - Calculate traveled-time between loc_0 to loc_i (tt_s) - Calculate traveled time (tt) - Loop every item in I_c, where $I_{c=0}$ is the first item and $I_{c=n}$ is the last item. - Scan item (I_c) in $loc_i \rightarrow loc_{(i,I_c)}$ - Record stamp-time - Calculate elapsed-time to pick and put item I_c - prev_loc = loc_i - next_loc = $loc_{(i+1)}$ - Calculate performance	
*Traveled time is approximated based on previous data and current item locations. *Performance is calculated based on current value of tt and approximated(average/mean) value of tt	

Architecture

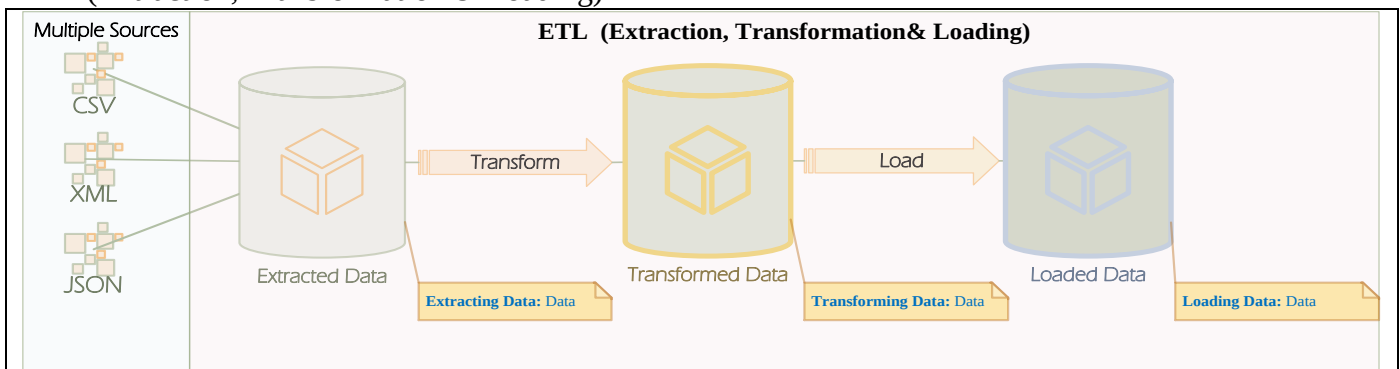
Entity Relational Diagram



Data Preparation



ETL (Extraction, Transformation & Loading)



ETL systems integrate data from multiple applications, typically supported by different vendors. The disparate systems containing the original data are frequently operated by different users. Extract, Transform and Load (ETL) refers to a process in database usage and especially in data warehousing that performs:

- Data extraction — extracts data from homogeneous or heterogeneous data sources
- Data transformation — transforms the data for storing it in the proper format for querying and analysis
- Data loading — loads it into the final target (operational data store/data warehouse)

Since the data extraction takes time, it is common to execute the three phases in parallel. While the data is being extracted, another transformation process executes. It processes the already received data and prepares it for loading. As soon as there is some data ready to be loaded into the target, the data loading kicks off without waiting for the completion of the previous phases.

ETL process pulls data out of source systems to place it into Synaptik database through the following tasks:

- Extraction

This is the most important aspect since extracting the data correctly sets the stage for the success of subsequent processes. This system combines data from different source systems, and each system may also use a different data format. Common data-source formats include relational databases, XML and flat files, but may also include non-relational database structures such as Information Management System (IMS) or other data structures such as Virtual Storage Access Method (VSAM) or Indexed Sequential Access Method (ISAM), or even formats fetched from outside sources by means such as web spidering or screen-scraping. In general, it aims to convert the data into a single format(json) appropriate for transformation processing. An intrinsic part of the extraction involves data validation to confirm whether the data pulled from the sources has the correct data type in a given domain. The data is rejected entirely, or in part, if it fails the validation rules. The rejected data is ideally reported back to the source system for further analysis. Sometimes, a data-validation rule is needed to accept data before next phase.

- Transform Data

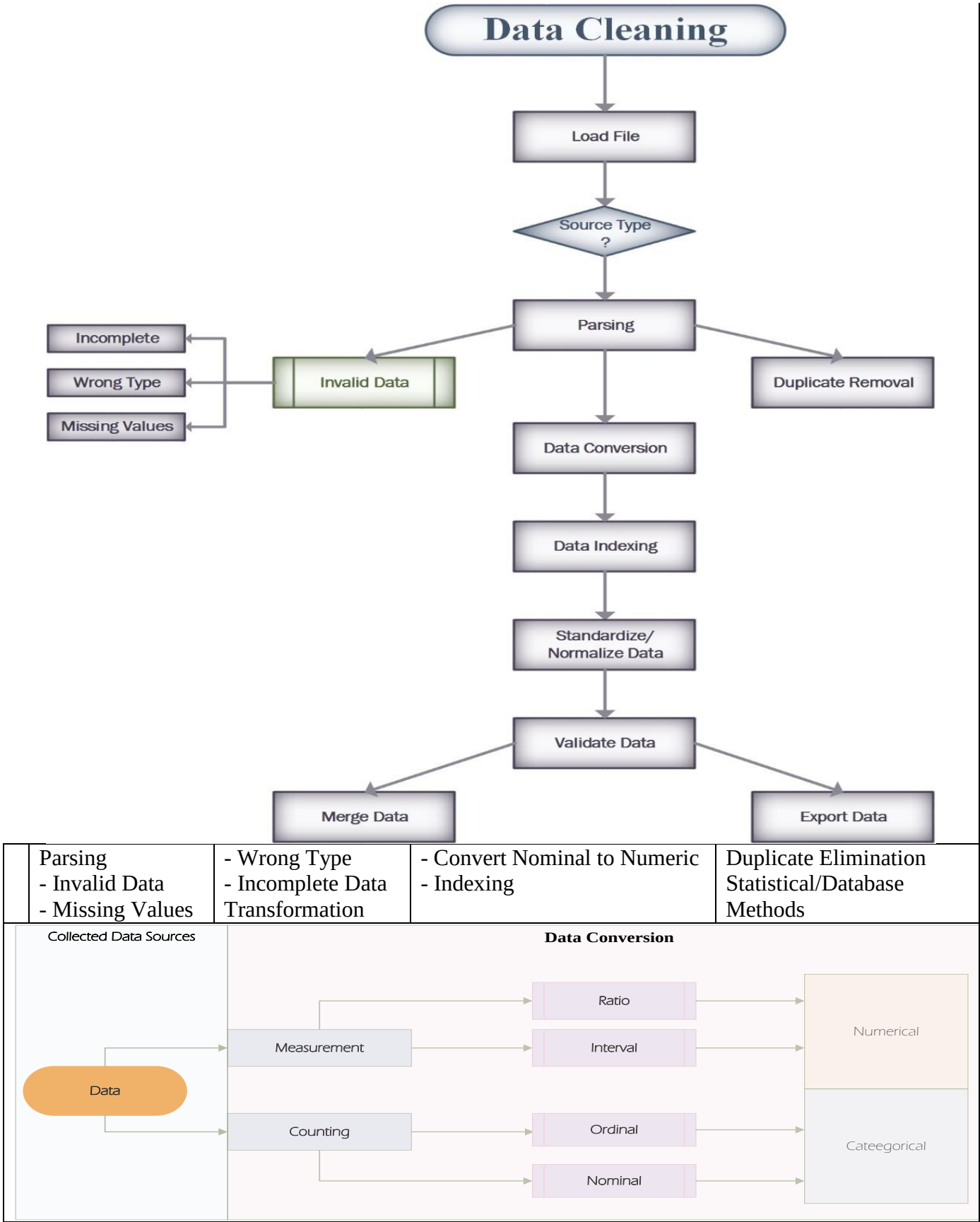
In this stage a series of rules are applied to the extracted data to “prepare” it for loading. Some data does not require any transformation, also known as "direct move" data. Cleaning the data is an important function of transformation that aims to pass only "proper" data. The relevant systems' interfacing and communicating is a challenge when different systems interact because of having different characters.

In some cases, the following transformation types may be required to meet the business and technical needs of the server:

- Selecting certain columns to load: (or selecting null columns not to load).
- Translating coded values: (e.g., system codes male as "1" and female as "2")
- Encoding free-form values: (e.g., mapping "Male" to "M")
- Deriving a new calculated value: (e.g., $\text{sale_amount} = \text{qty} * \text{unit_price}$)
- Sorting the data based on a list of columns to improve search performance
- Joining data from multiple sources (e.g., lookup, merge) and deduplicating the data
- Aggregating (for example, rollup — summarizing multiple rows of data — total sales for each store)
- Generating surrogate-key values
- Transposing or pivoting (turning multiple columns into multiple rows or vice versa)
- Splitting a column into multiple columns (e.g., converting a comma-separated list, etc.)
- Disaggregating repeating columns
- Looking up and validating the relevant data from tables or referential files
- Applying any form of data validation; failed validation may result in a full rejection of the data, etc.
- Load Data

This phase loads the data into the data warehouse. The data warehouses (or other parts of it) may add new data in a historical form at regular intervals. To understand this, consider a data warehouse that is required to maintain sales records of the last year. The entry of data for any one-year window is made in a historical manner. The timing and scope to append are strategic design choices dependent on the time available and the business needs. This system can maintain a history and audit trail of all changes to the data loaded in the data warehouse. As the load phase interacts with a database, the constraints defined in the database schema — as well as in triggers activated upon data load — apply (for example, uniqueness, referential integrity, mandatory fields), which also contribute to the overall data quality performance of the ETL process.

Data Cleaning



Basic operations include removing noise if appropriate, collect the necessary information to model or account for noise, deciding on strategies for handling missing data fields and account for time sequence information and known changes.

- Data Extraction

```
> pickpack <- read.csv("~/workspace/LTB-warehouse/pickpack.csv")
> str(pickpack)
'data.frame':    60561 obs. of  26 variables:
 $ ppDate.ppTime      : Factor w/ 19852 levels "4/11/2016 10:11",...: 313 313 313 314 314 314 315 316 317 317 ...
 $ User.ID            : Factor w/ 24 levels "ALEJANDROM","ALICIAM",...: 5 5 5 5 5 5 5 5 5 5 ...
 $ Quantity           : int  4 4 4 3 3 3 3 4 4 3 ...
 $ Zone              : Factor w/ 6 levels "", "200S", "BINSHELF",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ Area              : Factor w/ 5 levels "PIFLOORPP-Phase 1 - Floor Piece Pick",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ Travel.Sequence    : num  101000000000000 101000000000000 101000000000000 101000000000000 101000000000000 ...
 $ Storage.Location   : Factor w/ 5072 levels "2110530201","2110530202",...: 2324 2324 2324 2324 2585 2585 2585 2585 3198 3198 ...
 $ aisle             : Factor w/ 19 levels "", "211", "212",...: 12 12 12 13 13 13 13 14 14 14 ...
 $ bay              : int  53 53 53 13 13 13 13 55 55 55 ...
 $ Level            : Factor w/ 13 levels "", "1", "2", "3",...: 8 8 8 8 8 8 8 9 9 9 ...
 $ position         : Factor w/ 9 levels "", "1", "2", "A",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ OrderDate.OrderTime : Factor w/ 2371 levels "3/28/2016 6:48",...: 857 857 857 857 857 857 857 857 857 ...
 $ Order.Number      : int  804062448 804062448 804062448 804062448 804062448 804062448 804062448 804062448 ...
 $ Order.Line       : int  70 70 70 60 60 60 60 100 100 100 ...
 $ Part.Number      : Factor w/ 4597 levels "109B6400","14001-17",...: 4496 4496 4496 4422 4422 4422 4422 4425 4425 ...
 $ Work.Reference    : Factor w/ 58055 levels "WRR0407475","WRR0407477",...: 58 58 58 54 54 54 68 69 69 69 ...
 $ Shipment.ID       : Factor w/ 35872 levels "SID2284248","SID2285926",...: 21 21 21 21 21 21 21 21 21 21 ...
 $ shipped_date.shipped_time : Factor w/ 1902 levels "4/11/2016 11:20",...: 47 47 47 47 47 47 47 47 47 47 ...
 $ Carrier.Code      : Factor w/ 57 levels "AAFES PRIVATE FLEET",...: 35 35 35 35 35 35 35 35 35 35 ...
 $ car_type          : Factor w/ 3 levels "EXP", "GROUND",...: 3 3 3 3 3 3 3 3 3 3 ...
 $ Bill.To.Customer  : Factor w/ 236 levels "100151","100207",...: 73 73 73 73 73 73 73 73 73 73 ...
 $ Address.Name      : Factor w/ 236 levels "A HEAVENLY CELEBRATION",...: 10 10 10 10 10 10 10 10 10 10 ...
 $ early_ship_date.early_ship_time : Factor w/ 162 levels "", "1/12/2016 0:00",...: 116 116 116 116 116 116 116 116 116 116 ...
 $ late_ship_date.late_ship_time : Factor w/ 104 levels "", "10/21/2016 0:00",...: 88 88 88 88 88 88 88 88 88 88 ...
 $ ext_date.ext_time : Factor w/ 3 levels "", "4/13/2016 0:00",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ calc_pri         : int  9 9 9 9 9 9 9 9 9 9 ...
```

- Raw Data Analysis

```
> summary(pickpack)

ppDate.ppTime      User.ID      Quantity      Zone
5/20/2016 11:22:   244      NOELC      : 8381      Min.: 1.000000      : 1
5/20/2016 11:21:   226      HECTORJR : 8134      1st Qu.: 1.000000      200S : 31427
5/20/2016 11:26:   226      STEVEN  : 7922      Median: 1.000000      BINSHELF: 5913
5/20/2016 11:23:   223      STEPHANIAG: 6046      Mean: 2.042205      FLOOR : 2159
5/20/2016 11:24:   219      AUDELIAZ : 4991      3rd Qu.: 2.000000      FLOW123: 19432
5/20/2016 11:25:   210      NADIAS   : 3336      Max.: 213.000000      TOWER : 1629
(Other)           :59213      (Other) :21751

Area      Travel.Sequence      Storage.Location
PIFLOORPP-Phase 1 - Floor Piece Pick : 2159      Min.: 2110640201      TWREPL004 : 709
PIFLOWPP - Phase 1 Piece Pick Flow Bins :47795      1st Qu.:1010000000000000      TWREPL003 : 584
PIOVERFLPP - Phase 1 Flow Bin Overflow : 1      Median:1010000000000000      PIFLOORPP95 : 307
PIPALLETTP - Phase 1 Pallet Level Storage Piece Pick: 3064      Mean:115277902887000      FL002004A : 251
PITOWERPP-Phase I - Tower Piece Pick : 7542      3rd Qu.:1010000000000000      FL003004B : 204
NA's :2159      Max.:3050000000000000      PIFLOORPP100: 175
(Other) :58331

aisle      bay      Level      position      OrderDate.OrderTime      Order.Number
FL001 : 8046      Min.: 1.000000      B :11029      :51584      5/18/2016 10:41: 1504      Min.: 804029477
FL002 : 6197      1st Qu.: 32.000000      A :10843      1 : 1829      4/13/2016 13:43: 328      1st Qu.:804120039
FL216 : 5596      Median: 56.000000      C : 9978      A : 1289      4/22/2016 5:42 : 317      Median :804170291
FL003 : 5189      Mean: 57.59347      D : 6514      2 : 1235      4/22/2016 5:47 : 302      Mean :804169327
FL217 : 4548      3rd Qu.: 76.000000      E : 4911      E : 1031      5/24/2016 14:15: 189      3rd Qu.:804219127
FL220 : 4021      Max.:1030.000000      F : 4520      B : 1021      5/24/2016 14:13: 187      Max.: 804276047
(Other):26964      NA's :1630      (Other):12766      (Other): 2572      (Other) :57734

Order.Line      Part.Number      Work.Reference      Shipment.ID      shipped_date.shipped_time
Min.: 10.0000      5114325: 537      WRR0652985: 11      SID2327437: 1501      5/23/2016 12:22: 1501
1st Qu.: 10.0000      5106253: 426      WRR0758414: 10      SID2304272: 564      4/29/2016 13:41: 564
Median : 10.0000      5173172: 426      WRR0407475: 8      SID2320146: 386      5/13/2016 14:39: 388
Mean : 76.6632      5168060: 418      WRR0407489: 8      SID2304271: 315      4/29/2016 14:08: 316
3rd Qu.: 30.0000      5081298: 337      WRR0510657: 8      SID2298118: 292      4/22/2016 12:52: 305
Max.: 3870.0000      5050669: 279      WRR0627485: 8      SID2307555: 206      5/31/2016 18:15: 209
(Other):58138      (Other):60508      (Other): 57297      (Other) :57278

Carrier.Code      car_type      Bill.To.Customer      Address.Name
UPS GROUND :40191      EXP : 2599      151638 :12140      MIKASA.COM - DROP SHIP :12140
FEDEX HOME DELIVERY : 3941      GROUND:48529      155438 : 6654      PFALTZGRAFF.COM DROP SHIP: 6654
USPS PRIORITY MAIL : 2712      TL-LTL: 9433      155559 : 5506      KOHLS.COM -WHOLESALE DS : 5506
EMPLOYEE SALE : 1577      140950 : 5317      WAYFAIR LLC : 5317
FED EX GROUND : 1332      104827 : 3224      DILLARD DEPARTMENT STORES: 3224
Overflow to Consolidator: 1302      105763 : 3204      MACY S.COM FONTANA : 3204
(Other) : 9506      (Other):24516      (Other) :24516

early_ship_date.early_ship_time      late_ship_date.late_ship_time      ext_date.ext_time      calc_pri
5/17/2016 0:00: 2467      :50953      :60532      Min.: 0.000000
5/30/2016 0:00: 2046      5/6/2016 0:00 : 1143      4/13/2016 0:00: 25      1st Qu.:9.000000
5/9/2016 0:00 : 1606      6/11/2016 0:00: 869      5/6/2016 0:00 : 4      Median :9.000000
5/23/2016 0:00: 1438      10/5/2016 0:00: 322      Mean :8.613761
5/2/2016 0:00 : 1419      5/28/2016 0:00: 306      3rd Qu.:9.000000
4/11/2016 0:00: 1338      5/14/2016 0:00: 282      Max.: 9.000000
(Other) :50247      (Other) : 6686
```

Data Cleaning

- Data Attributes

```
> names(picknpack)
```

[1] "ppDate.ppTime"	"User.ID"	"Quantity"
[4] "Zone"	"Area"	"Travel.Sequence"
[7] "Storage.Location"	"aisle"	"bay"
[10] "Level"	"position"	"OrderDate.OrderTime"
[13] "Order.Number"	"Order.Line"	"Part.Number"
[16] "Work.Reference"	"Shipment.ID"	"shipped_date.shipped_time"
[19] "Carrier.Code"	"car_type"	"Bill.To.Customer"
[22] "Address.Name"	"early_ship_date.early_ship_time"	"late_ship_date.late_ship_time"
[25] "ext_date.ext_time"	"calc_pri"	

- Data Integration

```
> df <- read.csv("~/workspace/LTB-warehouse/datar1ts.csv")
> str(df)
'data.frame':    60561 obs. of  47 variables:
 $ pm      : int  4 4 4 4 4 4 4 4 4 4 ...
 $ pd      : int 11 11 11 11 11 11 11 11 11 11 ...
 $ py      : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ ppTime  : num 6.09 6.09 6.09 6.12 6.12 6.12 6.13 6.14 6.15 6.15 ...
 $ UserID  : int 1 1 1 1 1 1 1 1 1 1 ...
 $ Quantity: int 4 4 4 3 3 3 3 4 4 3 ...
 $ Zone    : int 1 1 1 1 1 1 1 1 1 1 ...
 $ Area     : int 1 1 1 1 1 1 1 1 1 1 ...
 $ TravelSequence : num 1.01e+14 1.01e+14 1.01e+14 1.01e+14 1.01e+14 1.01e+14 ...
 $ StorageLocation : int 1 1 1 2 2 2 2 3 3 3 ...
 $ aisle    : int 1 1 1 2 2 2 2 3 3 3 ...
 $ bay      : int 53 53 53 13 13 13 13 55 55 55 ...
 $ Level    : int 1 1 1 1 1 1 1 2 2 2 ...
 $ position : int 1 1 1 1 1 1 1 1 1 1 ...
 $ Om       : int 4 4 4 4 4 4 4 4 4 4 ...
 $ Od       : int 6 6 6 6 6 6 6 6 6 6 ...
 $ Oy       : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ OrderTime : num 8.11 8.11 8.11 8.11 8.11 8.11 8.11 8.11 8.11 8.11 ...
 $ OrderNumber : int 1 1 1 1 1 1 1 1 1 1 ...
 $ OrderLine  : int 70 70 70 60 60 60 60 100 100 100 ...
 $ PartNumber : int 1 1 1 2 2 2 2 3 3 3 ...
 $ WorkReference : int 1 1 1 2 2 2 3 4 4 4 ...
 $ ShipmentID  : int 1 1 1 1 1 1 1 1 1 1 ...
 $ sm          : int 4 4 4 4 4 4 4 4 4 4 ...
 $ sd          : int 12 12 12 12 12 12 12 12 12 12 ...
 $ sy          : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ shipped_time : num 12.6 12.6 12.6 12.6 12.6 ...
 $ CarrierCode  : int 1 1 1 1 1 1 1 1 1 1 ...
 $ car_type     : int 1 1 1 1 1 1 1 1 1 1 ...
 $ Bill.ToCustomer : int 135930 135930 135930 135930 135930 135930 135930 135930 135930 ...
 $ AddressName  : int 1 1 1 1 1 1 1 1 1 1 ...
 $ em           : int 4 4 4 4 4 4 4 4 4 4 ...
 $ ed           : int 5 5 5 5 5 5 5 5 5 5 ...
 $ ey           : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ early_ship_time : num 12.6 12.6 12.6 12.6 12.6 ...
 $ lm           : int 7 7 7 7 7 7 7 7 7 7 ...
 $ ld           : int 15 15 15 15 15 15 15 15 15 15 ...
 $ ly           : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ late_ship_time : num 12.6 12.6 12.6 12.6 12.6 ...
 $ em.1         : int 4 4 4 4 4 4 4 4 4 4 ...
 $ ed.1         : int 12 12 12 12 12 12 12 12 12 12 ...
 $ ey.1         : int 2016 2016 2016 2016 2016 2016 2016 2016 2016 2016 ...
 $ ext_time     : num 12.6 12.6 12.6 12.6 12.6 ...
 $ calc_pri     : int 6 6 6 6 6 6 6 6 6 6 ...
 $ cart_time    : num 11.2 11.2 11.2 11.2 11.2 ...
 $ performance_rate : num 0.15 0.15 0.15 0.15 0.15 0.15 0.15 0.15 0.15 0.15 ...
 $ performance  : num 89 89 89 89 89 ...
```

Data indexing was only performed for analysis purpose to observe the validity and uniqueness of the data. The current data is only worth two months and a half of data with incomplete weeks and days data. Therefore, this data is not very useful for illustration of advanced features that requires big data for analysis purpose.

Sample Data Structure

Item Class

```
# Object to represent items with reference to its pick and put time
#-----
class Item (object):
    def __init__(self, loc, time):
        self.location      = loc
        self.pick_time     = time
        self.count         = 1
        self.prep_build_time = 0
        self.area          = ""
        self.zone          = ""
        self.ordered_date  = ""
        self.shipped_date  = ""

    def increment(self):
        self.count += 1

    def update_qty(self, item_qty):
        self.count += int(item_qty)

    def set_time(self, time):
        self.pick_time = time

    def get_time(self):
        return self.pick_time

    def get_count(self):
        return self.count
```

Tour Class

```
# Object to represent a manifest tour with reference to its items
#-----
class Tour (object):
    def __init__(self, date, tkey):
        self.date          = date
        self.start_time     = date.time()
        self.end_time       = date.time()
        self.items          = defaultdict(list)
        self.time           = 0
        self.performance_rate = 0
        self.performance    = 0
        self.count          = 1
        self.work_magnitude = 0
        self.utilized_resources = 0
        self.tour_id        = tkey;

    def increment(self):
        self.count += 1

    def add_items(self, key, item):
        self.items[key].append(item)

    def set_items(self, items):
        self.items = items
        #self.start_time = items[items.keys()[0]].pick_time
        #self.end_time   = items[items.keys()[-1]].pick_time
        #sorted(self.items)      # sort by cartprep start time

    def get_items(self):
        return self.items

    def get_count(self):
        return self.count
```

User Class

```
# Object to represent a user, with reference to its tours and items.
#-----
class User (object):
    def __init__(self, user_id):
        self.tour_time           = 0
        self.avg_tours_time      = 0
        self.tours_list          = defaultdict(list)
        self.tour_performance_rate = 0
        self.tour_performance    = 0
        self.avg_tour_time       = 0
        self.count               = 1
        self.total_work_magnitude = 0
        self.total_utilized_resources = 0
        self.areas_list          = defaultdict(list)
        self.zones_list          = defaultdict(list)
        self.tours_number        = 0
        self.user_id             = user_id

    def increment(self):
        self.count += 1

    def get_count(self):
        return self.count

    def add_tours(self, key, tour):
        self.tours_list[key] = tour
        self.tours_number += 1

    def get_avg_tours_time(self):
        return self.avg_tours_time/self.tours_number
        #sorted(self.tours_list)      # sort by cartprep start time

    def get_tours_list(self):
        return self.tours_list
```

Company Class

```
# Object to represent a company with reference to its users, their cartpreps and items.
#-----
class Company (object):
    def __init__(self):
        self.overall_avg_tours_time      = 0
        self.overall_avg_tours_performance_rate = 0
        self.overall_avg_tours_performance    = 0
        self.users_list                   = defaultdict(list)
        self.count                         = 1
        self.user_counter                 = 0
        self.areas_list                   = defaultdict(list)
        self.zones_list                   = defaultdict(list)

    def increment(self):
        self.count += 1

    def get_count(self):
        return self.count

    def set_users_list(self, users_list):
        self.users_list = users_list
        self.users_counter = len(users_list)
        #sorted(self.users_list)      # sort by cartprep start time

    def get_users_list(self):
        return self.users
```

Data Warehouse User Interface (UI)

Framework Layout (Matt)	User Authentication (Greg)
• Sign-In (<i>P</i>) • Dashboard • SOWs • Warehouse App (<i>P</i>) ◦ Dashboard	• Users Credential (<i>P</i>) ◦ User Table in MongoDB ◦ Integration of User Table with Synaptik • Users Permissions ◦ Field(s)/Application(s) ▪ Example: Sales, Ecommerce, Marketing, Etc. ◦ Data ▪ Is it an Admin? • Knowledge Discovery ◦ Basic Data Analysis (<i>P</i>) ◦ Data Mining Tasks (Learning) ◦ Advanced Data Mining Tasks (Deep Learning)
MongoDB setup (Greg & Matt)	
• Database integration with Synaptik • Data Ingest (<i>P</i>) ◦ Data Access (Goes in hand with User's Permission) ◦ Data Export • Set a database for each application clean data ◦ Reference Employee (for all employees within this company) ◦ Reference Customer (for all customers within this company) ◦ Reference Product (for all products within this company) • Set a database for Synaptik ◦ Contain a reference to all applications ◦ Client Table (all clients) ◦ Employee Table (all employees for Data Analysis on Marketing) ◦ Customer Table (all employees for Data Analysis on Marketing)	
Data Analysis (Piter)	Data Visualization (Greg & Piter)
• Clean Data (<i>P</i>) ◦ Check for Anomalies • Data Integration (<i>P</i>) ◦ Calculate New Fields ◦ Integrate New Fields • Transform Data (<i>P</i>) ◦ Convert to Json format • Knowledge Discovery ◦ Basic Analysis (<i>P</i>) ◦ Data Mining (DM) Techniques ◦ Advanced DM Techniques	• Dashboard ◦ Overall Performance ◦ Current Year Performance • Application(s) • Dashboard (Warehouse is Priority) ◦ Future insight – C Customized Dashboard? ◦ Current Year Tour Time Statistics ◦ Overall Performance ◦ Current Year Performance • Knowledge Discovery ◦ Basic Analysis (<i>P</i>) ◦ Data Mining (DM) Techniques ◦ Advanced DM Techniques